

Security Model

AirOne — Team AirOne. This document describes the threat model, the controls that are actually implemented (each one has a test), the operator bootstrap procedure, and the residual risks that are **deliberately left open** and must be handled by the deployment.

1. Threat model

Asset	Threats considered	Where handled
Operator accounts / API	credential stuffing, brute force, default passwords, token theft/replay, privilege escalation	\$2, \$3
REST API	hostile bodies, unknown fields, oversized requests, cross-origin abuse, information disclosure	\$4
Radio telemetry (LoRa)	spoofed or replayed frames, malformed headers, hostile JSON payloads, provenance forgery	\$5
Stored data / secrets	secret leakage into config/images/logs, world-readable database, tampered audit log	\$6, \$7
Deployment	running as root, writable image, baked secrets, exposed ports	\$8
Supply chain	known-vulnerable dependencies	\$9

Out of scope (see §10): confidentiality of the radio link, TLS termination, physical access to the ground-station host.

2. Accounts and passwords

- **No accounts ship with the software.** The user table is empty until an administrator creates one with the launcher (§3). The former demo accounts (admin/AirOneAdmin!2026, viewer/viewer123) are only seeded when AIRONE_ALLOW_DEFAULT_USERS=1 is set, are flagged `is_default_account`, and are forced to change their password on first login (HTTP 403 PASSWORD_CHANGE_REQUIRED until POST `/api/v1/auth/change-password`).
- Passwords are hashed with **bcrypt** (`security.bcrypt_rounds`, default 12). Plain passwords are never stored, logged or audited.
- **Password policy** (`src/security/passwords.py`): at least 12 characters, one lowercase, one uppercase, one digit and one symbol, not on the

deny-list (which includes the former demo passwords), not containing the username.

- Accounts can be **disabled** (POST /api/v1/users/<name>/disable); a disabled account can neither log in nor refresh.
- **Login throttling** (src/security/lockout.py): after security.max_login_attempts failures (default 5, 15-minute window) the account is locked for lockout_seconds (default 300 s) with exponential back-off on repeated lockouts (capped at 1 h); a per-source-IP cap (max_login_attempts_per_ip, default 20) limits enumeration. Locked requests return **429** with Retry-After and data.state = ACCOUNT_LOCKED / IP_LOCKED. Wrong username and wrong password return the same generic 401 body.
- Failed logins, lockouts, password changes and user management are all audited (§7).

3. Bootstrap procedure

Secrets come only from the environment (see .env.example); they are never accepted on the command line and never written to YAML.

1. Signing secret (>= 32 chars). The process refuses to start without it.

```
export AIRONE_JWT_SECRET="$(python3 -c 'import secrets;  
print(secrets.token_urlsafe(48))')"
```

2. Create the first administrator (password from env or generated once).

```
AIRONE_ADMIN_PASSWORD='<strong password>' python3 launcher.py --create-admin
```

or: python3 launcher.py --create-admin --generate-password # printed ONCE

3. Further accounts / maintenance

```
python3 launcher.py --create-user ops1 --role OPERATOR --generate-password
```

```
python3 launcher.py --reset-password ops1 --must-change # uses
```

```
AIRONE_ADMIN_PASSWORD
```

```
python3 launcher.py --list-users
```

```
python3 launcher.py --verify-audit # checks logs/audit.jsonl
```

4. Run

```
python3 launcher.py --validate-only # warns if no account exists
```

```
python3 launcher.py
```

--generate-password always sets must_change_password, so the printed password is single-use. Valid roles: VIEWER OPERATOR SCIENTIST ENGINEER ADMIN.

Environment variables

Variable	Required	Purpose
AIRONE_JWT_SECRET	yes	JWT signing key (>= 32 chars;

Variable	Required	Purpose
AIRONE_ADMIN_PASSWORD	bootstrap only	placeholders refused) Password for --create-admin, --create-user, --reset-password
AIRONE_LINK_KEY	optional	Hex link key (>= 16 bytes) for telemetry frame authentication (\$5)
AIRONE_API_HOST	optional	Overrides api.host; default loopback 127.0.0.1
AIRONE_ALLOW_DEFAULT_USERS	dev only	1 seeds the demo accounts (must-change)
AIRONE_ALLOW_CORS_WILDCARD	dev only	1 permits cors_origins: ["*"]
AIRONE_GUI_USER / AIRONE_GUI_PASSWORD	GUI only	GUI login; without them the GUI runs UNAUTHENTICATED (read-only public status)
AIRONE_SERIAL_PORT	optional	Serial device override

4. Tokens and API hardening

JWT (src/security/auth.py)

- Algorithm pinned to **HS256** (alg: none, HS512 and any other algorithm are rejected); claims iss=airone, aud=airone-api, type (access/refresh), jti, iat, exp, sub, role are all required.
- Lifetimes: security.access_token_minutes (default 15, hard cap 1440) and security.refresh_token_days (default 7, hard cap 30). Larger configured values are clamped.
- **Refresh rotation:** POST /auth/refresh revokes the presented refresh token and issues a new pair; reuse of a rotated token is refused.
- **Persistent revocation:** POST /auth/logout and rotation store the jti in the revoked_tokens table, so revocation survives restarts; expired entries are pruned.
- Access tokens are refused where a refresh token is expected and vice versa.

Routes (src/api/)

- All request bodies are validated by strict marshmallow schemas — **unknown fields are rejected** (400), identifiers must match `^[A-Za-z0-9_-]{1,64}$`, sizes/limits are bounded.
- `MAX_CONTENT_LENGTH` = `api.max_body_bytes` (default 256 KiB) → 413.
- **CORS** is restricted to `api.cors_origins`; "*" is refused unless `AIRONE_ALLOW_CORS_WILDCARD=1` (logged as development-only).
- **Response headers** on every response: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Content-Security-Policy: default-src 'none'; frame-ancestors 'none', Referrer-Policy: no-referrer, Cross-Origin-Resource-Policy: same-origin, Permissions-Policy, Cache-Control: no-store for `auth/user/config` routes, Server: AirOne (the WSGI server no longer advertises Werkzeug/Python versions).
- X-Request-ID is echoed only if it is a short (≤ 64) `[A-Za-z0-9_-]` token; anything else is replaced by a fresh UUID.
- Errors return a generic envelope; tracebacks and internal paths are never returned. Flask DEBUG is forced off.
- `PUT /api/v1/config` only accepts runtime-mutable keys (`mission_id`, `objectives`, `default_primary`, `default_secondary`, `pipeline`, `scientific`, `ml`, `gui`, `simulation`, `logging`); `api`, `security`, `storage`, `telemetry` are refused (400) and secret-looking keys are redacted in GET.
- `GET /api/v1/telemetry/export` returns a sanitised **basename** (file), never a filesystem path.
- Rate limits (Flask-Limiter, `api.rate_limits`): login 10/min, refresh 30/min, export 5/min, ML train 2/h, config 20/min → 429 + audit `RATE_LIMIT_HIT`.

RBAC — roles are hierarchical (`VIEWER < OPERATOR < SCIENTIST < ENGINEER < ADMIN`). User management requires ADMIN; a valid token with an insufficient role yields 403, a missing/invalid token 401.

5. Telemetry link authentication and input hardening

The radio link is plain LoRa; anyone in range can transmit frames with the right MAGIC. Two layers address this:

Frame authentication (optional, recommended) —

`FLAG_AUTHENTICATED` (0x08). When a link key is configured, every frame carries an 8-byte tag = `HMAC-SHA256(key, header || payload)[0:8]`, placed between the payload and the CRC (`docs/telemetry_protocol.md`). The same construction is implemented in the firmware (`firmware/airone_cansat/airone_frame.h`, portable SHA-256/HMAC, `AIRONE_LINK_KEY_HEX`) and in `src/telemetry/protocol.py`; parity is tested against RFC 4231 vectors and byte-for-byte between C and Python.

Verification state is always explicit (ParsedPacket.auth_state, frame.metadata["auth_state"], and auth_mode in the TelemetryLink entry of the health report / receiver health status):

State	Meaning	Effect on data
NOT_CONFIGURED	no key on the ground station, no tag on the frame	accepted, unchanged
AUTHENTICATED	tag verified	accepted
UNAUTHENTICATED	key configured, frame has no tag	quality SUSPECT; INVALID if telemetry.require_authenticated_frames: true
UNVERIFIABLE	frame has a tag, ground station has no key	quality SUSPECT
INVALID_TAG	tag mismatch	frame reported as corrupted, no measurements

require_authenticated_frames: true without AIRONE_LINK_KEY is a startup error. The simulator signs its frames with the same key so --simulate exercises the identical path; simulated provenance is never upgraded by a valid tag (the tag proves origin, not realness).

Limits of this mechanism (see §10): the tag gives **integrity and origin authentication only** — no confidentiality; the tag is truncated to 64 bits; replay protection is by sequence tracking, not by the tag.

Parser and payload hardening

- Header pre-checks before any allocation: unknown VERSION, PACKET_TYPE, FLAGS bits or a PAYLOAD_LENGTH > 65535 are rejected and counted (invalid_version, invalid_type, invalid_flags, invalid_length); counters are exposed in the receiver health status.
- Sequence regression beyond the order tolerance is flagged replay (counted replays); duplicates are flagged, never silently dropped.
- Payload JSON: ≤ 16 KiB, ≤ 64 fields, nesting depth ≤ 3 , top level must be an object, keys/strings ≤ 64 chars. Violations record payload_error and produce **no measurements**.
- Values must be finite numbers; strings, booleans, NaN/inf become quality=INVALID, valid=False — never coerced into a plausible reading.
- A payload may declare itself SIMULATED (downgrade) but can never claim a higher provenance (CALIBRATED, etc.) — declared upgrades are ignored.
- Implausible timestamps (before 2020 or after 2100) are replaced by receive time and flagged timestamp_replaced.

6. Storage and secrets

- The SQLite database file is created with mode **0600**.
- .gitignore / .dockerignore exclude .env* (except .env.example), keys (*.key, *.pem, *.p12), secrets/, databases and runtime data.
- Secrets are read from the environment only; GET /api/v1/config redacts secret-looking keys; the launcher never accepts passwords as arguments.
- Trained ML artefacts are SHA-256 checksummed and refused when tampered (SafeLoadError, see ml_tier.md).

7. Audit chain

Every security-relevant event is appended to logs/audit.jsonl as a hash-chained JSON line (entry_checksum = SHA-256 of the entry including previous_checksum). The chain **continues across restarts** (the last checksum on disk is loaded at start). Events: LOGIN_SUCCESS, LOGIN_FAILURE, ACCOUNT_LOCKED, LOGOUT, TOKEN_REFRESH, TOKEN_REVOKE, PASSWORD_CHANGE, USER_CREATE, USER_DELETE, USER_DISABLE, USER_ENABLE, CONFIG_CHANGE, MISSION_STATE_CHANGE, MODEL_LOAD, MODEL_TRAIN, DATA_DELETE, DATA_EXPORT, SECURITY_VIOLATION, RATE_LIMIT_HIT, TELEMETRY_AUTH_FAILURE.

Verification detects edited entries, deleted/inserted entries and garbled lines, reporting the first bad line:

```
python3 launcher.py --verify-audit [logs/audit.jsonl] # exit 0 ok, 2 broken
python3 -m src.security.audit logs/audit.jsonl      # JSON result
```

8. Container / deployment posture

- Dockerfile: python:3.12-slim, dedicated user airone (uid 10001), no secret baked into the image, headless dependency set, healthcheck on /api/v1/status.
- docker-compose.yml: API published on **127.0.0.1:5000 only**, AIRONE_JWT_SECRET **required** from the environment (compose refuses to start without it), AIRONE_ALLOW_DEFAULT_USERS=0, read_only root filesystem with tmpfs /tmp, cap_drop: ALL, no-new-privileges, persistent volumes only for data/, logs/, models/.
- The API binds to loopback by default; binding to 0.0.0.0 requires an explicit AIRONE_API_HOST (a warning is logged) and a TLS reverse proxy.

9. Dependencies

pip-audit -r requirements.txt --disable-pip --no-deps on 2026-09-04: **No known vulnerabilities found** (pip-audit 2.10.1) after bumping PyJWT 2.13.0, cryptography 50.0.1, Flask 3.1.3, flask-cors 6.0.5, marshmallow 3.26.2, requests 2.34.2 and pytest 9.1.1. Re-run the command before every release.

10. Residual risks (deliberately not solved here)

Risk	Status	Mitigation expected from the deployment
No TLS — the API speaks plain HTTP	open by design	keep the loopback bind, or terminate TLS in a reverse proxy (nginx/caddy)
Radio link has no confidentiality	open	LoRa E22 AES option or accept that telemetry is public; the HMAC tag only prevents forgery
Auth tag truncated to 64 bits	accepted	keeps frames small; forgery needs $\sim 2^{63}$ attempts at LoRa data rates
Replay of authenticated frames	mitigated, not prevented	sequence regression is flagged replay; a fresh flight must not reuse sequence numbers
Lockout / rate-limit state is in memory	accepted	resets on restart; fine for a single ground station
GUI password in an environment variable	accepted	GUI is a local desktop client; do not share the shell environment

Risk	Status	Mitigation expected from the deployment
le Link key stored in firmw are sourc e (AIRO NE_LI NK_KE Y_HEX)	accepted	rotate per campaign; anyone with the flight-computer flash can read it
Audit log is tampe r- evidence nt , not tampe r- proof	accepted	archive audit.jsonl off-host regularly
Depen dency vulner abiliti es discov ered after 2026- 09-04	n/a	re-run pip-audit

11. Test coverage

Controls in this document are exercised by tests/test_security_hardening.py (passwords, user store, throttle, JWT, revocation), tests/test_api_security.py (routes, headers, CORS, schemas, logout, rotation, user management, config immutability), tests/test_telemetry_hardening.py (header pre-checks, HMAC states, hostile payloads, pipeline verdicts, simulator, audit chain, launcher bootstrap) and tests/test_firmware_frame_parity.py (C/Python frame and HMAC parity, RFC 4231 vectors).